

CSE 5095: Types and Programming Languages

Binding and Substitution

Stefan Muller

Sept. 15–17

1 Syntax and Intuition

Today we extend the syntax of **E** with variables and a construct called “let binding” which you may be familiar with from some programming languages. We’ll use ... in BNF syntax to mean “everything that was in this language before.”

$$\begin{array}{lcl} e & ::= & \dots \\ & | & x \quad \text{Variable} \\ & | & \text{let } x = e \text{ in } e \quad \text{Binding} \end{array}$$

A new expression might be something like

$$\text{let } x = e_1 \text{ in } e_2$$

The intuition (I’ll give rules later) is that we evaluate e_1 to a value and then use (*substitute*) that value for x in e_2 . We say “ x is *bound* in e_2 ” (but *not* in e_1 !) Conversely, the *scope* of x is e_2 .

For example:

$$\begin{aligned} & \text{let } x = \bar{1} + \bar{2} \text{ in } x + \bar{3} \\ \mapsto & \text{let } x = \bar{3} \text{ in } x + \bar{3} \\ \mapsto & \bar{3} + \bar{3} \\ \mapsto & \bar{6} \end{aligned}$$

We can introduce multiple instances of the same variable with overlapping scopes. A variable refers to its nearest enclosing binding:

$$\text{let } \textcolor{red}{x} = \bar{1} \text{ in } (\text{let } x = \bar{2} \text{ in } \textcolor{blue}{x} + \bar{1})$$

Here, the x in the final expression refers to the second (inner) binding, the one bound to the value $\bar{2}$. Outside of the scope of the inner binding, x still refers to the *outer* x .

$$\text{let } \textcolor{red}{x} = \bar{1} \text{ in } (\text{let } x = \bar{2} \text{ in } \textcolor{blue}{x} + \bar{1}) + \textcolor{red}{x}$$

This matters a lot because let binding **isn’t** state update! (If you’ve programmed in a functional language, you’re aware of this distinction; if you’ve only programmed in imperative languages like Java or Python, it might take some getting used to.)

$$\begin{aligned} & \text{let } \textcolor{red}{x} = \bar{1} \text{ in } (\text{let } x = \bar{2} \text{ in } \textcolor{blue}{x} + \bar{1}) + \textcolor{red}{x} \\ \mapsto & (\text{let } \textcolor{blue}{x} = \bar{2} \text{ in } \textcolor{blue}{x} + \bar{1}) + \bar{1} \\ \mapsto & (\bar{2} + \bar{1}) + \bar{1} \\ \mapsto^2 & \bar{4} \end{aligned}$$

2 Free and Bound Identifiers

If a variable isn't *bound*, it's *free*. Actually, freeness generally turns out to be a more useful notion and we'll define it on its own. Let $FV(e)$ mean the set of "free variables" of e .

$$\begin{aligned}
FV(x) &= \{x\} \\
FV(\bar{n}) &= \emptyset \\
FV("s") &= \emptyset \\
FV(e_1 + e_2) &= FV(e_1) \cup FV(e_2) \\
FV(e_1 \wedge e_2) &= FV(e_1) \cup FV(e_2) \\
FV(|e|) &= FV(e) \\
FV(\text{let } x = e_1 \text{ in } e_2) &= FV(e_1) \cup (FV(e_2) \setminus \{x\})
\end{aligned}$$

For example, in $\text{let } x = y \text{ in } x + \bar{2}$, x is bound (by the let binding) and y is free (in this expression; if this is part of a larger program, it may be bound by some outer let).

A key property of bound identifiers/variables is that their names don't matter. We can always consistently rename them through a process called *alpha* (α)-conversion. Two expressions are α -equivalent if they're the same up to α -conversion.

$$\text{let } x = \bar{1} \text{ in } x + \bar{2} \equiv_\alpha \text{let } y = \bar{1} \text{ in } y + \bar{2}$$

but:

$$\text{let } x = y \text{ in } x + \bar{2} \not\equiv_\alpha \text{let } y = z \text{ in } y + \bar{2}$$

Note that this only works if the new name doesn't already appear free in its scope:

$$\text{let } y = \bar{1} \text{ in } x \not\equiv_\alpha \text{let } x = \bar{1} \text{ in } x$$

but

$$\text{let } y = \bar{1} \text{ in } x \equiv_\alpha \text{let } z = \bar{1} \text{ in } x$$

3 Dynamic Semantics

3.1 Substitution

Before we define the operational semantics of let formally, we need to define the substitution operation. We write $[e_1/x]e_2$ as the result of substituting e_1 for all *free* instances of x in e_2 . It's defined inductively on e_2 .

$$\begin{aligned}
[e/x]x &= e \\
[e/x]y &= y & y \neq x \\
[e/x]\bar{n} &= \bar{n} \\
[e/x]"s" &= "s" \\
[e/x](e_1 + e_2) &= [e/x]e_1 + [e/x]e_2 \\
[e/x](e_1 \wedge e_2) &= [e/x]e_1 \wedge [e/x]e_2 \\
[e/x]|e_0| &= |[e/x]e_0| \\
[e/x]\text{let } x = e_1 \text{ in } e_2 &= \text{let } x = [e/x]e_1 \text{ in } e_2 \\
[e/x]\text{let } y = e_1 \text{ in } e_2 &= \text{let } y = [e/x]e_1 \text{ in } [e/x]e_2 \quad y \neq x, y \notin FV(e)
\end{aligned}$$

To see why we need $y \notin FV(e)$ in that last definition, consider

$$[x + \bar{2}/y](\text{let } x = \bar{1} \text{ in } y + \bar{2})$$

The x in the expression we're substituting in is free and presumably refers to some x bound elsewhere in the program. But if we ignore that restriction on the last line of the definition and substitute to get

$$\text{let } x = \bar{1} \text{ in } (x + \bar{2}) + \bar{2}$$

The x we substituted in is now bound. We say it was “captured.” The form of substitution definition we have above is usually referred to as “capture-avoiding substitution.” So what do we do if $y \in \text{FV}(e)$? We can α -convert y so it isn’t!

$$\begin{aligned} & [x + \bar{2}/y](\text{let } x = \bar{1} \text{ in } y + \bar{2}) \\ \equiv_{\alpha} & [x + \bar{2}/y](\text{let } z = \bar{1} \text{ in } y + \bar{2}) \\ = & \text{let } z = \bar{1} \text{ in } (x + \bar{2}) + \bar{2} \end{aligned}$$

3.2 Operational Semantics

Now we can define the step rules for let bindings. We can actually define two versions. The “call-by-value” or “strict” or “eager” version evaluates e_1 to a value before substituting:

$$\frac{e_1 \mapsto e'_1}{\text{let } x = e_1 \text{ in } e_2 \mapsto \text{let } x = e'_1 \text{ in } e_2} \text{ (STEPSEARCHLET)} \qquad \frac{e_1 \text{ val}}{\text{let } x = e_1 \text{ in } e_2 \mapsto [e_1/x]e_2} \text{ (STEPLET)}$$

The “call-by-name” or “lazy” version substitutes immediately.

$$\frac{}{\text{let } x = e_1 \text{ in } e_2 \mapsto [e_1/x]e_2} \text{ (STEPLETCBN)}$$

Both versions work (and, as we’ll see shortly in the context of a different language), arrive at the same final answer. They might take different numbers of steps though.

Consider two extreme cases:

$$\text{let } x = \bar{1} + \bar{1} + \bar{1} + \bar{1} + \bar{1} \text{ in } \bar{2}$$

and

$$\text{let } x = \bar{1} + \bar{1} + \bar{1} + \bar{1} + \bar{1} \text{ in } x + x + x + x + x$$

Note that a variable by itself is neither a value nor takes a step. This is because by the time we’re looking at it, a variable should have been substituted away. If not, e.g., in the expression $x + \bar{1}$, it’s an *unbound variable*, which should be a compile error.

4 Statics

It’s not enough to add new typing rules for variables and let bindings, we actually need to change the judgment to add a *context* Γ , which maps variables to their types. We write a context as a comma-separated list of mappings like this: $x_1 : \tau_1, \dots, x_n : \tau_n$. We can thus write $\Gamma, x : \tau$ to mean Γ extended with a binding of x to τ . Usually, we will only write this if $x \notin \text{Dom}(\Gamma)$ to avoid introducing duplicate variables. We can generally ensure this with α -conversion.

We’ll write $\bullet$ for the empty context (sometimes you’ll see $\emptyset$).

With that, here are the rules:

$$\begin{array}{c} \frac{\Gamma(x) = \tau}{\Gamma \vdash x : \tau} \text{ (TYPEVAR)} \qquad \frac{}{\Gamma \vdash \bar{n} : \text{int}} \text{ (TYPENUM)} \qquad \frac{}{\Gamma \vdash \text{“}s\text{”} : \text{string}} \text{ (TYPESTRING)} \\[10pt] \frac{\Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 + e_2 : \text{int}} \text{ (TYPEADD)} \qquad \frac{\Gamma \vdash e_1 : \text{string} \quad \Gamma \vdash e_2 : \text{string}}{\Gamma \vdash e_1 \wedge e_2 : \text{string}} \text{ (TYPECAT)} \\[10pt] \frac{\Gamma \vdash e : \text{string}}{\Gamma \vdash |e| : \text{int}} \text{ (TYPELEN)} \qquad \frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \text{let } x = e_1 \text{ in } e_2 : \tau_2} \text{ (TYPELET)} \end{array}$$

(Note that the rules for existing contexts are essentially the same as before, just with the context threaded through.)

4.1 Structural Properties

There are some important properties of the typing judgment.

Lemma 1 (Exchange/Permutation). *If $\Gamma_1, x : \tau_x, y : \tau_y, \Gamma_2 \vdash e : \tau$ and $x \neq y$, then $\Gamma_1, y : \tau_y, x : \tau_x, \Gamma_2 \vdash e : \tau$.*

Proof. By induction on the derivation of $\Gamma_1, x : \tau_x, y : \tau_y, \Gamma_2 \vdash e : \tau$.

- **TYPEVAR.** Then $e = z$ and $(\Gamma_1, x : \tau_x, y : \tau_y, \Gamma_2)(z) = \tau$. We haven't changed the bindings, so $(\Gamma_1, y : \tau_y, x : \tau_x, \Gamma_2)(z) = \tau$. Apply **TYPEVAR**.
- **TYPERNUM.** Apply **TYPERNUM**.
- **TYPEADD.** Then $e = e_1 + e_2$ and $\tau = \text{int}$ and $\Gamma_1, x : \tau_x, y : \tau_y, \Gamma_2 \vdash e_1 : \text{int}$ and $\Gamma_1, x : \tau_x, y : \tau_y, \Gamma_2 \vdash e_2 : \text{int}$. By induction, $\Gamma_1, y : \tau_y, x : \tau_x, \Gamma_2 \vdash e_1 : \text{int}$ and $\Gamma_1, y : \tau_y, x : \tau_x, \Gamma_2 \vdash e_2 : \text{int}$. Apply **TYPEADD**.
- **TYPELET.** Then $e = \text{let } z = e_1 \text{ in } e_2$ and $\Gamma_1, x : \tau_x, y : \tau_y, \Gamma_2 \vdash e_1 : \tau_1$ and $\Gamma_1, x : \tau_x, y : \tau_y, \Gamma_2, z : \tau_1 \vdash e_2 : \tau$. By induction, $\Gamma_1, y : \tau_y, x : \tau_x, \Gamma_2 \vdash e_1 : \tau_1$ and $\Gamma_1, y : \tau_y, x : \tau_x, \Gamma_2, z : \tau_1 \vdash e_2 : \tau$. Apply **TYPELET**.

□

Because this holds, we can say that the order of bindings in a context doesn't matter and permute contexts freely. As a result, you'll sometimes see **TYPEVAR** written this way:

$$\frac{}{\Gamma, x : \tau \vdash x : \tau} \text{ (TYPEVAR)}$$

We can also add a variable to the context and not change the typing properties.

Lemma 2 (Weakening). *If $\Gamma \vdash e : \tau$ and $x \notin \text{Dom}(\Gamma)$, then $\Gamma, x : \tau' \vdash e : \tau$.*

Proof. By induction on the derivation of $\Gamma \vdash e : \tau$.

- **TYPEVAR.** Then $e = z$ and $\Gamma(z) = \tau$. Because $x \notin \text{Dom}(\Gamma)$, we have $x \neq z$, and so $(\Gamma, x : \tau')(z) = \tau$. Apply **TYPEVAR**.
- **TYPERNUM.** Apply **TYPERNUM**.
- **TYPEADD.** Then $e = e_1 + e_2$ and $\tau = \text{int}$ and $\Gamma \vdash e_1 : \text{int}$ and $\Gamma \vdash e_2 : \text{int}$. By induction, $\Gamma, x : \tau' \vdash e_1 : \text{int}$ and $\Gamma, x : \tau' \vdash e_2 : \text{int}$. Apply **TYPEADD**.
- **TYPELET.** Then $e = \text{let } z = e_1 \text{ in } e_2$ and $\Gamma \vdash e_1 : \tau_1$ and $\Gamma, z : \tau_1 \vdash e_2 : \tau$. By induction, $\Gamma, x : \tau' \vdash e_1 : \tau_1$ and $\Gamma, z : \tau_1, x : \tau' \vdash e_2 : \tau$. By exchange, $\Gamma, x : \tau', z : \tau_1 \vdash e_2 : \tau$. Apply **TYPELET**.

□

Note that, because of exchange, we can put the new variable anywhere in the context and not just at the end even though the lemma is stated just for new variables at the end.

There's another one we won't really use and won't prove.

Lemma 3 (Contraction). *If $\Gamma, x : \tau, x : \tau \vdash e : \tau$, then $\Gamma, x : \tau \vdash e : \tau$.*

These three properties (and a couple others) are sometimes called “structural properties”. While they hold in almost all of the type systems we'll see in this class, it can sometimes be useful to define *substructural* type systems in which they don't! We may see some later in the class.

Another important structural property is substitution:

Lemma 4 (Substitution). *If $\Gamma, x : \tau' \vdash e : \tau$ and $\Gamma \vdash e' : \tau'$, then $\Gamma \vdash [e'/x]e : \tau$.*

Proof. By induction on the derivation of $\Gamma, x : \tau' \vdash e : \tau$.

- **TYPEVAR.** We have two cases:
 - $e = x$ (and therefore $\tau = \tau'$). Then $[e'/x]e = e'$ and we have $\Gamma \vdash e' : \tau'$ by assumption.
 - $e = y \neq x$. Then $(\Gamma, x : \tau')(y) = \tau$. We have $[e'/x]e = y$. Because $y \neq x$, we have $\Gamma(y) = \tau$. Apply **TYPEVAR**.
- **TYPERNUM** and **TYPESTRING** just apply the same rule again, as in the previous proofs.
- **TYPEADD**, **TYPECAT**, and **TYPELEN** are pretty straightforward inductions.
- **TYPELET.** Then $e = \text{let } y = e_1 \text{ in } e_2$ and $\Gamma, x : \tau' \vdash e_1 : \tau_1$ and $\Gamma, x : \tau', y : \tau_1 \vdash e_2 : \tau$. By α -conversion, we can assume $y \neq x$ and $y \notin \text{FV}(e')$. So $[e'/x]e = \text{let } y = [e'/x]e_1 \text{ in } [e'/x]e_2$. By induction, $\Gamma \vdash [e'/x]e_1 : \tau_1$. By weakening, $\Gamma, y : \tau_1 \vdash e' : \tau'$. By exchange, $\Gamma, y : \tau_1, x : \tau' \vdash e_2 : \tau$. By induction, $\Gamma, y : \tau_1 \vdash [e'/x]e_2 : \tau$. Apply **TYPELET**.

□

5 Type Safety

We've added new typing and step rules (and actually changed the typing judgment itself), so we need to prove Progress and Preservation for the new cases. Because we only evaluate *closed* expressions, which are defined as expressions with no free variables, the expressions we're stepping should always be well-typed with an empty context. But preservation turns out to be true for any context.

We'll prove type safety for the call-by-value (CBV) rules, but you can prove them similarly for CBN.

Theorem 1 (Preservation). *If $\Gamma \vdash e : \tau$ and $e \mapsto e'$, then $\Gamma \vdash e' : \tau$.*

Proof. By induction on the derivation of $e \mapsto e'$. Despite the addition of the context to the typing rule, the cases we already proved don't really change, so we'll just prove the new cases.

- **STEPSEARCHLET.** Then $e = \text{let } x = e_1 \text{ in } e_2$ and $e' = \text{let } x = e'_1 \text{ in } e_2$ and $e_1 \mapsto e'_1$. By inversion on **TYPELET**, we have $\Gamma \vdash e_1 : \tau'$ and $\Gamma, x : \tau' \vdash e_2 : \tau$. By induction, $\Gamma \vdash e'_1 : \tau'$. Apply **TYPELET**.
- **STEPLET.** Then $e = \text{let } x = e_1 \text{ in } e_2$ and $e' = [e_1/x]e_2$. By inversion on **TYPELET**, we have $\Gamma \vdash e_1 : \tau'$ and $\Gamma, x : \tau' \vdash e_2 : \tau$. By Substitution, $\Gamma \vdash [e_1/x]e_2 : \tau$.

□

Progress, on the other hand, is only true for an empty context (think about why).

Theorem 2 (Progress). *If $\bullet \vdash e : \tau$, then $e \text{ val}$ or there exists e' such that $e \mapsto e'$.*

Proof. By induction on the derivation of $\bullet \vdash e : \tau$. Again, the existing cases don't change much.

- **TYPEVAR.** Then $e = x$ and $\Gamma(x) = \tau$, but $\Gamma = \bullet$, so this is a contradiction.
- **TYPELET.** Then $e = \text{let } x = e_1 \text{ in } e_2$ and $\bullet \vdash e_1 : \tau'$ and $x : \tau' \vdash e_2 : \tau$. By induction, either $e_1 \text{ val}$ or $e_1 \mapsto e'_1$.
 - $e_1 \text{ val}$. Then apply **STEPLET**.
 - $e_1 \mapsto e'_1$. Then apply **STEPSEARCHLET**.

□